

INTRODUCTION

Mobile application, also referred to as a mobile app or simply an app, is a computer program or software application designed to run on a mobile device such as a phone, tablet or watch. Mobile applications often stand in contrast to desktop applications which are designed to run on desktop computers, and web applications which run in mobile web browsers rather than directly on the mobile device.

Apps were originally intended for productivity assistance such as email, calendar, and contact databases, but the public demand for apps caused rapid expansion into other areas such as mobile games, factory automation, GPS and location-based services, order-tracking, and ticket purchases, so that there are now millions of apps available. Many apps require Internet access. Apps are generally downloaded from app stores, which are a type of digital distribution platforms.

The term "app", short for "software application", has since become very popular; in 2010, it was listed as "Word of the Year" by the American Dialect Society.

Apps are broadly classified into three (3) types: **native apps**, **hybrid** and **cross-platform**.

iOS vs. Android

Globally, most people's mobile devices run on iOS and Android. In 2003, when the smartphone market was still in its early development stages, Android. Inc. was founded in Palo Alto, California, with the initial intention of creating an operating system for digital cameras. Two years later, the company was acquired by Google, confirming rumors of the tech giant's plans to enter the mobile phone market. The first Android operating system was introduced in 2007, with the goal to compete with the then-leading smartphone operating system Symbian. Android accomplished this goal by 2011, when it became the leader of the global smartphone OS market, with about 47 percent of the share, while Symbian fell to third place and was pushed out of the market by 2013.

As Symbian faded out of the market, Apple became Android's main competitor on the mobile operating system market. However, Android has been the main player in the market since 2012, and is forecast to maintain this leadership in the coming years. In the first quarter of 2016, about 84 percent of smartphones sold to end users used Android as its operating system. Android is expected to maintain its smartphone market share of about 85 percent up to 2020,

while Apple's iOS is projected to account for 15 percent. One of the main reasons for Android's popularity is its strong collaboration with smartphone manufacturers, such as Sony, Samsung and HTC, while the iOS is limited to operating on Apple devices.

Upon its success in the smartphone market, Android has expanded its operating system to tablets. Android had a slow start in this market, mainly due to the lack of quality applications for Android tablets. However, Android managed to overcome this issue and establish itself as the leading tablet operating system in 2012. Android is now the most used operating system for tablets worldwide, with a 67 percent share of the global market in 2015.

Native App

A native app, or native application, is a software application built in a specific programming language, for the specific device platform, either iOS or Android. Native iOS apps are written in Swift or Objective-C and native Android apps are written in Java or Kotlin.

Apple and Google offer app developers their own development tools, interface elements and standardised SDK; Xcode and Android Studio. These tools allow any professional developer to develop a native mobile app relatively easily.

Advantages of Native Apps

1. Fast and Responsive native apps offer the fastest, most reliable and most responsive experience to users.
2. Easy to tap into wider device functionality native makes tapping into the camera, microphone, compass, accelerometer and swipe gestures super easy.
3. Match app UI/UX to platform conventions you're more likely to please your users due to the way you can match each app's UI/UX to the platform conventions.
4. Native Apps Are More Secure.

Disadvantages of Native Apps

1. Cost: The overall cost involved in the development and maintenance of a native app is considerably higher. This is due to the fact that there should be separate versions of the same application
2. Development: Developing a native app is a difficult process since separate developers are needed for each platform. For an example, different developers must be hired to develop Android and IOS version of the same application.

3. **Time Consumption:** Since native apps are developed for multiple platforms, it requires more time.

Hybrid App

A hybrid app is a software application that combines elements of both native apps and web applications. Hybrid apps are essentially web apps that have been put in a native app shell. Once they are downloaded from an app store and installed locally, the shell is able to connect to whatever capabilities the mobile platform provides through a browser that's embedded in the app. Similar to Web apps, hybrid apps are typically written in HTML5, CSS and JavaScript. Hybrid apps run code inside a container. The device's browser engine is used to render HTML and JavaScript and native APIs to access device-specific hardware.

Advantages of hybrid apps include:

1. Will operate on different platforms.
2. Faster build times compared to native apps.
3. Cheaper to develop compared to building two versions of a native app for two different platforms.

Disadvantages of hybrid apps include:

1. A limited user interface
2. Difficulty or impossibility to exploit the platforms' capacities in full.
3. Slower performance and transition between pages.
4. Dependency on the browser's speed.

Cross-platform

Cross-platform software is a type of software application that works on multiple operating systems or devices, which are often referred to as platforms. Cross-platform app development shares the same codebase as hybrid development. Yes, mobile developers use the same native elements to create the app, but they differ in providing a great user experience with different UI.

The goal of cross-platform app development is to target different operating systems with one project. You create these apps using cross-platform frameworks, which use platform-specific SDKs (Android SDKs and iOS SDKs) from a unified API. This enables you to easily access the different platform SDKs and libraries.

Private companies create these frameworks. Examples of popular cross-platform frameworks include:

- **React Native by Meta.** It uses JavaScript as the programming language.
- **Flutter by Google.** It uses Dart as the programming language.
- **Xamarin by Microsoft** (which is being migrated to MAUI). It uses C# and XAML as the programming language.

Cross-platform mobile apps are usually compiled to use native UI elements that make the app feel native.

Cross-platform tools

To deploy your app on Android, you still need to have Android SDK installed and a Windows, macOS, or Linux machine. For iOS, you still need to have iOS SDK, Xcode, and exclusively a macOS machine.

Advantages of Cross-platform

Creating cross-platform mobile applications can benefit you in the following ways:

1. **Low costs:** Cross-platform mobile development needs only a single team of developers. They need to be knowledgeable in the chosen framework. The app reaches a wider audience to start gaining traction and testing the market.
2. **Code reusability:** Cross-platform frameworks allow you to use a single codebase. This ensures consistency in the business logic used.
3. **Rapid development:** By reusing code and improving productivity, you get to market faster. Cross-platform frameworks have also optimized the process of app testing during development.
4. **Easier maintenance:** Updating a cross-platform app is easier. This is because you are dealing with a single codebase. All builds targeting the different platforms use the same code, ensuring consistency.

Disadvantages of Cross-platform

Cross-platform mobile development addresses some of the disadvantages that native app development faces. Yet, it has its fair share of disadvantages:

1. **Larger digital footprint:** Cross-platform apps are usually larger. You need to pay attention to the mobile optimization options available.
2. **Difficult integrations:** Cross-platform frameworks do not integrate all platform-specific features. Some hardware-specific integrations like using GPUs might need skills in native app development.
3. **Lower performance:** Cross-platform frameworks usually bundle a custom runtime to run your app. It is responsible for communicating with platform-specific services. This adds another layer of computation, leading to lower performance.
4. **Delayed platform features:** New SDK releases typically include new features and updates. With cross-platform frameworks, you have to wait for a separate update to access these newer features.

Native or Cross-platform apps: Which is best?

The decision to develop a native or cross-platform app is crucial. There are various scenarios that benefit native over cross-platform apps and vice-versa. Carefully considering your audience, budget, development timeline, performance and security needs, and so on will help in the decision-making process.

Let's look at a few areas to consider when determining which development method to use.

Time to market

This is a widespread concern for start-ups and new product lines. You want to ship the product early to start getting quality feedback from users. Cross-platform app development would be the best fit for this scenario, as it is quick to build and iterate. Native mobile development would be time-consuming and harder to maintain, which ultimately makes the time to market slower.

Security

You need to consider the reputation of the business and the impact that losing users' trust would have. The risks can be very high for certain types of mobile applications, such as a mobile banking app. In these scenarios, native mobile development would be a better route. It provides many built-in security features, including file encryption and intelligent fraud detection using specific OS libraries. While cross-platform mobile development is ideal for getting the app to market faster, native apps provide superior security, stability, and scalability.

Performance

Mobile apps can have high compute requirements especially for games. For apps that need accelerated performance, native mobile development is often the best fit. Using cross-platform frameworks in this scenario would require extra effort and native app expertise. Optimizing performance for a specific operating system enables you to ensure your application runs as efficiently and effectively as possible.

Development costs

Some companies have higher budgets than others to develop mobile apps. For lower budgets, go for cross-platform apps, as you only need a small team. Additionally, cross-platform development allows you to keep costs in check by reusing your code and projects.

Flutter

According to Google “Flutter is an open source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase.”

Flutter a simple framework based on Dart language, provides high performance by rendering the UI directly in the operating system’s canvas rather than through native framework. Flutter also offers many ready to use widgets (UI) to create a modern application. These widgets are optimized for mobile environment and designing the application using widgets is as simple as designing HTML. To be specific, Flutter application is itself a widget. Flutter widgets also supports animations and gestures. The application logic is based on reactive programming. Widget may optionally have a state. By changing the state of the widget, Flutter will automatically (reactive programming) compare the widget’s state (old and new) and render the widget with only the necessary changes instead of re-rendering the whole widget.

Features of Flutter

Flutter framework offers the following features to developers:

1. Modern and reactive framework.
2. Uses Dart programming language and it is very easy to learn.
3. Fast development.
4. Beautiful and fluid user interfaces.
5. Huge widget catalogue.

6. Runs same UI for multiple platforms.
7. High performance application.
8. Open-Source
9. Cross-platform
10. Hot Reload

Advantages of Flutter

Flutter comes with beautiful and customizable widgets for high performance and outstanding mobile application. It fulfils all the custom needs and requirements. Besides these, Flutter offers many more advantages as mentioned below:

1. Dart has a large repository of software packages which lets you to extend the capabilities of your application.
2. Developers need to write just a single codebase for both applications (both Android and iOS platforms).
3. Flutter needs lesser testing. Because of its single code base, it is sufficient if we write automated tests once for both the platforms.
4. Flutter's simplicity makes it a good candidate for fast development. Its customization capability and extendibility make it even more powerful.
5. With Flutter, developers have full control over the widgets and its layout.
6. Flutter offers great developer tools, with amazing hot reload.

Disadvantages of Flutter

Despite its many advantages, flutter has the following drawbacks in it:

1. Since it is coded in Dart language, a developer needs to learn new language (though it is easy to learn).
2. Modern framework tries to separate logic and UI as much as possible but, in Flutter, user interface and logic is intermixed. We can overcome this using smart coding and using high level module to separate user interface and logic.
3. Flutter is yet another framework to create mobile application. Developers are having a hard time in choosing the right development tools in hugely populated segment.

Flutter Installation

To install flutter visit <https://docs.flutter.dev/get-started/install> and select your respective OS

Widgets in Flutter

Flutter is Google's UI toolkit for crafting beautiful, natively compiled iOS and Android apps from a single code base. To build any application we start with widgets the building block of flutter applications. Widgets describe what their view should look like given their current configuration and state. It includes a text widget, row widget, column widget, container widget, and many more. Each element on a screen of the Flutter app is a widget, the view of the screen completely depends upon the choice and sequence of the widgets used to build the app. And the structure of the code of an app is a tree of widgets.

Category of Widgets:

There are mainly 14 categories in which the flutter widgets are divided. They are mainly segregated on the basis of the functionality they provide in a flutter application.

1. **Accessibility:** These are the set of widgets that make a flutter app more easily accessible.
2. **Animation and Motion:** These widgets add animation to other widgets.
3. **Assets, Images, and Icons:** These widgets take charge of assets such as display images and show icons.
4. **Async:** These provide async functionality in the flutter application.
5. **Basics:** These are the bundle of widgets which are absolutely necessary for the development of any flutter application.
6. **Cupertino:** These are the ios designed widgets.
7. **Input:** This set of widgets provide input functionality in a flutter application.
8. **Interaction Models:** These widgets are here to manage touch events and route users to different views in the application.
9. **Layout:** This bundle of widgets helps in placing the other widgets on the screen as needed.
10. **Material Components:** These are set of widgets that mainly follow material design by Google.
11. **Painting and effects:** These are the set of widgets which apply visual changes to their child widgets without changing their layout or shape.

12. **Scrolling:** This provides scroll ability to a set of other widgets that are not scrollable by default.
13. **Styling:** This deals with the theme, responsiveness, and sizing of the app.
14. **Text:** This display text.

Types of Widgets:

There are broadly two types of widgets in the flutter namely:

1. Stateless Widget
2. Stateful Widget

Stateless Widget: The widgets whose state can not be altered once they are built are called stateless widgets. These widgets are immutable once they are built i.e any amount of change in the variables, icons, buttons, or retrieving data can not change the state of the app.

Stateful Widgets: The widgets whose state can be altered once they are built are called stateful Widgets. These states are mutable and can be changed multiple times in their lifetime. This simply means the state of an app can change multiple times with different sets of variables, inputs, data.

MaterialApp class

MaterialApp Class: MaterialApp is a predefined class in a flutter. It is likely the main or core component of flutter. We can access all the other components and widgets provided by Flutter SDK. Text widget, Dropdownbutton widget, AppBar widget, Scaffold widget, ListView widget, StatelessWidget, StatefulWidget, IconButton widget, TextField widget, Padding widget, ThemeData widget, etc. are the widgets that can be accessed using MaterialApp class. There are many more widgets that are accessed using MaterialApp class. Using this widget, we can make an attractive app.

Scaffold class

Scaffold is a class in flutter which provides many widgets or we can say APIs like Drawer, SnackBar, BottomNavigationBar, FloatingActionButton, AppBar, etc. Scaffold will expand or occupy the whole device screen. It will occupy the available space. Scaffold will provide a framework to implement the basic material design layout of the application.

AppBar Widget

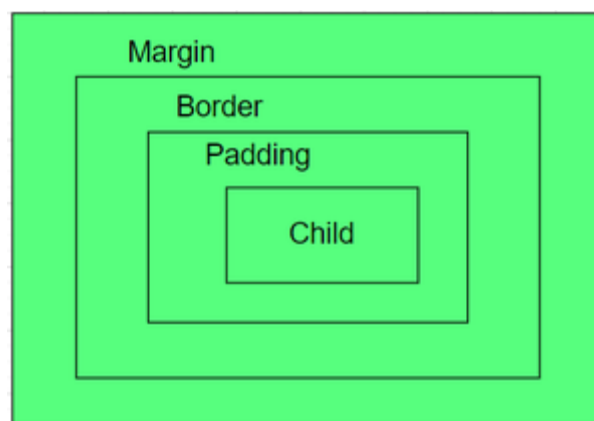
AppBar is usually the topmost component of the app (or sometimes the bottom-most), it contains the toolbar and some other common action buttons. As all the components in a flutter application are a widget or a combination of widgets. So AppBar is also a built-in class or widget in flutter which gives the functionality of the AppBar out of the box. The AppBar widget is based on Material Design and much of the information is already provided by other classes like MediaQuery, Scaffold as to where the content of the AppBar should be placed. Though the AppBar class is very flexible and can be easily customized, we can also use the SilverAppBar widget which gives scrollable functionality to the app bar. Or we can create our own custom app bar from scratch.

Row and Column Widgets

Row and Column are the two most important and powerful widgets in Flutter. These widgets let you align children horizontally and vertically as per the requirement. As we know that when we design any UI (User Interface) in flutter, we need to arrange its content in the Row and Column manner so these Row and Column widgets are required when designing UI.

Container class

Container class in flutter is a convenience widget that combines common painting, positioning, and sizing of widgets. A Container class can be used to store one or more widgets and position it on the screen according to our convenience. Basically, a container is like a box to store contents.



Properties of Container Class:

1. **child:** Container widget has a property 'child:' which stores its children. The child class can be any widget. Let us take an example, taking a text widget as a child.

2. **color:** The color property sets the background color of the entire container. Now we can visualize the position of the container using a background color.
3. **height and width:** By default, a container class takes the space that is required by the child. We can also specify height and width to the container based on our requirements.
4. **margin:** The margin is used to create an empty space around the container. Observe the white space around the container. Here `EdgeInsets.geometry` is used to set the margin. `.all()` indicates that margin is present in all four directions equally.
5. **padding:** The padding is used to give space from the border of the container to its children.
6. **alignment:** The alignment is used to position the child within the container. We can align in different ways: bottom, bottom center, left, right, etc.
7. **decoration:** The decoration property is used to decorate the box (e.g., give a border). This paints behind the child. Whereas `foregroundDecoration` paints in front of a child. Let us give a border to the container. But both color and border color cannot be given.
8. **transform:** This property of container helps us to rotate the container.

Note: There can be many more operations that can be performed to container class. Moreover, the container class is used often while developing flutter applications.